SỞ GIÁO DỤC VÀ ĐÀO TẠO
TỈNH LÂM ĐỒNG

ĐỀ THI THỬ HỌC SINH GIỎI CẤP TỈNH
ĐỀ SỐ 06 - LỚP 12 THPT

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

KỲ THI CHỌN HỌC SINH GIỎI
Môn thi: TIN HỌC (Lập trình Python)
Thời gian làm bài: 90 phút

CẤU TRÚC DANH MỤC BÀI THI

Bài	Tên bài	Tệp mã nguồn	Tệp vào	Tệp ra	Điểm
1	TỔNG CÁC SỐ ĐỐI XỨNG TRONG XÂU	`PALINNUM.PY`	`PALINNUM.INP`	`PALINNUM.OUT`	10.0
2	ĐÓNG GÓI RAU CỦ ĐÀ LẠT	`RAUCU.PY`	`RAUCU.INP`	`RAUCU.OUT`	10.0

BÀI 1: TỔNG CÁC SỐ ĐỐI XỨNG TRONG XÂU (10.0 điểm)

- Tên tệp mã nguồn: `PALINNUM.PY`
- Tệp dữ liệu vào: `PALINNUM.INP`
- Tệp kết quả ra: `PALINNUM.OUT`

Cho một xâu ký tự S có độ dài không quá 100.000 ký tự bao gồm các chữ cái tiếng Anh (chữ hoa, chữ thường) và các chữ số. Hãy viết chương trình bóc tách tất cả các số nguyên dương xuất hiện liên tiếp trong xâu S.
Một số nguyên dương X được gọi là "Số đối xứng" (Palindrome number) nếu giá trị của X khi đọc từ trái sang phải hay từ phải sang trái đều giống nhau (ví dụ: các số 121, 5, 88, 12321 là số đối xứng; các số 120, 45 không phải là số đối xứng).
Yêu cầu: Hãy đếm số lượng các số đối xứng tìm được, in ra danh sách các số đó theo đúng thứ tự xuất hiện trong xâu S và tính tổng của tất cả các số đối xứng này.

Dữ liệu vào (Tệp PALINNUM.INP):
- Một dòng duy nhất chứa xâu ký tự S.

Kết quả ra (Tệp PALINNUM.OUT):
- Dòng 1: Ghi một số nguyên là số lượng các số đối xứng tìm được (Nếu không tìm thấy số đối xứng nào, ghi -1).
- Dòng 2: Ghi danh sách các số đối xứng tìm được theo đúng thứ tự xuất hiện (cách nhau bởi 1 dấu cách).
- Dòng 3: Ghi một số nguyên là tổng của tất cả các số đối xứng tìm được.

Ví dụ:

PALINNUM.INP	PALINNUM.OUT
abc121xyz5k88m120	3 121 5 88 214

Ràng buộc kỹ thuật (Subtasks):
- Subtask 1 (60% số điểm): Các số nguyên tách ra từ xâu có giá trị <= 10^3.
- Subtask 2 (40% số điểm): Các số nguyên tách ra từ xâu có giá trị <= 10^9.

BÀI 2: ĐÓNG GÓI RAU CỦ ĐÀ LẠT (10.0 điểm)

- Tên tệp mã nguồn: `RAUCU.PY`
- Tệp dữ liệu vào: `RAUCU.INP`
- Tệp kết quả ra: `RAUCU.OUT`

Một hợp tác xã nông sản sạch tại thành phố Đà Lạt cần vận chuyển các túi rau củ xuất khẩu lên một thùng xe tải. Hợp tác xã có N túi rau củ được thu hoạch trong ngày, túi thứ i có khối lượng là A[i] (kg).
Thùng xe tải vận chuyển có tải trọng chở tối đa là M (kg). Để tối ưu không gian và số lượng đóng gói, ban quản lý quyết định ưu tiên xếp các túi rau củ có khối lượng nhỏ lên xe trước nhằm xếp được số lượng túi rau củ nhiều nhất có thể.
Yêu cầu: Hãy xác định số lượng túi rau củ tối đa chở được lên xe và khối lượng còn trống (chưa dùng tới) của thùng xe tải.

Dữ liệu vào (Tệp RAUCU.INP):
- Dòng 1: Chứa 2 số nguyên dương N và M (1 <= N <= 10^5, 1 <= M <= 10^9).
- Dòng 2: Chứa N số nguyên dương A[1], A[2], ..., A[N] đại diện cho khối lượng từng túi rau củ (A[i] <= 10^9).

Kết quả ra (Tệp RAUCU.OUT):
- Một dòng duy nhất chứa 2 số nguyên cách nhau bởi 1 khoảng trắng lần lượt là: Số lượng túi rau củ chở được và Khối lượng còn trống của thùng xe tải.

Ví dụ:

RAUCU.INP	RAUCU.OUT
5 60 25 10 30 15 40	3 10

Giải thích: Chọn 3 túi rau củ có khối lượng 10, 15, 25 kg. Tổng khối lượng = 50 kg <= 60 kg. Khối lượng còn trống = 60 - 50 = 10 kg.

Ràng buộc kỹ thuật (Subtasks):
- Subtask 1 (50% số điểm): N <= 10^3, M <= 10^6.
- Subtask 2 (50% số điểm): N <= 10^5, M <= 10^9.

ĐÁP ÁN VÀ HƯỚNG DẪN CHẤM ĐỀ SỐ 06

1. HƯỚNG DẪN BÀI 1: TỔNG CÁC SỐ ĐỐI XỨNG TRONG XÂU (PALINNUM.PY)

Phân tích thuật toán:

- Bước 1: Đọc xâu S từ tệp PALINNUM.INP.
- Bước 2: Bóc tách các cụm chữ số liên tiếp bằng cách duyệt qua từng ký tự của S. Nếu ký tự là chữ số (char.isdigit()), nối vào biến tạm `temp`. Khi gặp ký tự không phải chữ số (hoặc kết thúc xâu), nếu `temp` không rỗng thì chuyển thành số nguyên và thêm vào mảng `numbers`.
- Bước 3: Định nghĩa hàm kiểm tra số đối xứng `is_palindrome(n)` bằng cách so sánh xâu `str(n)` với xâu đảo ngược `str(n)[::-1]`. Nếu `str(n) == str(n)[::-1]` thì n là số đối xứng.
- Bước 4: Lọc danh sách các số đối xứng, đếm số lượng và tính tổng bằng hàm `sum()`. Xuất kết quả ra tệp PALINNUM.OUT.

Mã nguồn Python tham khảo (PALINNUM.py):

```
def is_palindrome(n):
    s_num = str(n)
    return s_num == s_num[::-1]

# Đọc dữ liệu từ file PALINNUM.INP
with open("PALINNUM.INP", "r", encoding="utf-8") as f:
    s = f.read().strip()

# Bóc tách các số nguyên từ xâu S
temp = ""
numbers = []
for char in s:
    if char.isdigit():
        temp += char
    else:
        if temp != "":
            numbers.append(int(temp))
            temp = ""
if temp != "":
    numbers.append(int(temp))

# Lọc các số đối xứng
palindromes = [x for x in numbers if is_palindrome(x)]

# Ghi kết quả ra file PALINNUM.OUT
with open("PALINNUM.OUT", "w", encoding="utf-8") as f:
    if not palindromes:
        f.write("-1\n")
    else:
        f.write(f"{len(palindromes)}\n")
        f.write(" ".join(map(str, palindromes)) + "\n")
        f.write(f"{sum(palindromes)}\n")
```

2. HƯỚNG DẪN BÀI 2: ĐÓNG GÓI RAU CỦ ĐÀ LẠT (RAUCU.PY)

Phân tích thuật toán (Tham lam - Greedy):

- Bước 1: Đọc N, M và mảng A gồm khối lượng N túi rau củ từ tệp RAUCU.INP.
- Bước 2: Để chở được số lượng túi rau củ nhiều nhất, ta áp dụng thuật toán Tham lam (Greedy): Sắp xếp mảng A theo thứ tự tăng dần bằng `a.sort()`. Khối lượng nhỏ nhất sẽ được xếp trước.
- Bước 3: Duyệt qua từng túi rau củ đã sắp xếp. Nếu tổng khối lượng hiện tại cộng thêm khối lượng túi đang xét <= M thì chọn túi đó, tăng biến đếm `count` và cộng khối lượng vào `total_weight`.

Ngược lại thì dừng lặp.
- Bước 4: Khối lượng còn trống = `M - total_weight`. Xuất `count` và `M - total_weight` ra tệp RAUCU.OUT.

Mã nguồn Python tham khảo (RAUCU.py):

```
# Đọc dữ liệu từ file RAUCU.INP
with open("RAUCU.INP", "r", encoding="utf-8") as f:
    lines = f.read().split()

if lines:
    n = int(lines[0])
    m = int(lines[1])
    a = [int(x) for x in lines[2:]]

    # Chiến thuật tham lam: Sắp xếp khối lượng tăng dần
    a.sort()

    count = 0
    total_weight = 0

    for weight in a:
        if total_weight + weight <= m:
            total_weight += weight
            count += 1
        else:
            break

    free_space = m - total_weight

    # Ghi kết quả ra file RAUCU.OUT
    with open("RAUCU.OUT", "w", encoding="utf-8") as f:
        f.write(f"{count} {free_space}\n")
```